

Software Requirements Specification (SRS)

Young Fighters Academy — Cricket Academy Management System

Project	Young Fighters Academy (YFA) — Cricket Academy Management System
Version	1.0
Date	August 2026
Status	Approved
Source of truth	Live production codebase (repo Young Fighters Club, branch master)

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of the Young Fighters Academy (YFA) web application. It describes the complete feature set of the system as implemented in the production codebase: a public-facing academy website, a role-based management portal for academy staff, and self-service portals for students and parents.

The intended audience is:

- Academy administrators and stakeholders
- Development and maintenance teams
- QA and testing teams
- New contributors onboarding to the codebase

1.2 Scope

The system is a single web application (Next.js) deployed on Vercel, backed by a PostgreSQL (Neon) database. It covers:

- Public marketing website (about, programs, coaches, players, matches, gallery, contact, apply)
- Online admission applications with an internal review/conversion pipeline
- Student, batch, coach, and user management
- Attendance tracking (manual marking and QR-code scanning)
- Fee collection management and expense tracking
- Performance assessments, development goals, and player rankings
- Training session management
- Match management with full scorecards
- Report generation and export (Excel, PDF, CSV)
- Notifications, announcements, audit logs, and settings

- PWA capabilities: offline support, background sync, and push notifications

Out of scope:

- Mobile native applications (the web app is installable as a PWA)
- Online payment processing (payments are recorded manually; reminder messages are generated for WhatsApp)
- Public self-registration of accounts (users are created by the administrator)

1.3 Definitions and Abbreviations

Term	Definition
ADMIN	Administrator role with full system access
COACH	Coaching staff role with scoped management access
STUDENT	Player role with read-only access to own records
PARENT	Guardian role with read-only access to linked children's records
Batch	A training group with assigned coach, schedule, and capacity
YFA-NNNN	Auto-generated student ID (e.g., YFA-0012)
QR token	Unique per-student token encoded in a QR code for attendance scanning
MOTM	Man of the Match
PWA	Progressive Web App
SW	Service Worker
IDOR	Insecure Direct Object Reference (access-control vulnerability class)
RBAC	Role-Based Access Control
SRS	Software Requirements Specification
VAPID	Voluntary Application Server Identification (Web Push protocol)

1.4 References

Document / Artifact	Location
Database schema	prisma/schema.prisma
Route structure	src/app/
Server actions	src/app/actions/
API routes	src/app/api/
Middleware / access control	src/proxy.ts, src/lib/rbac.ts, src/lib/auth.ts, src/lib/session.ts
Seed data	prisma/seed.ts
Build configuration	next.config.ts
PWA manifest	public/manifest.webmanifest
Service worker	public/sw.js

2. Overall Description

2.1 Product Perspective

YFA is a standalone, cloud-hosted web application. It is not a module of a larger system. It replaces manual record keeping (paper registers, spreadsheets) at the cricket academy with a centralized digital platform that spans the entire student lifecycle — from public admission application, through enrollment and training, to match performance tracking.

The product has two distinct surfaces:

1. **Public website** — marketing and application pages visible to anyone.
2. **Authenticated portal** — role-specific dashboards and management modules.

2.2 Product Functions (summary)

#	Function	Primary Actors
F1	Public website content (landing, about, programs, coaches, players, matches, gallery, contact)	Public
F2	Online admission application + review/conversion pipeline	Public, ADMIN
F3	Authentication, password reset, profile management	All roles
F4	Role-based dashboards with KPIs, charts, and activity feeds	All roles
F5	Student management (create, edit, activate/deactivate, soft-delete, documents, photos)	ADMIN, COACH
F6	Batch management (create, edit, delete, assign coach, capacity checks)	ADMIN
F7	Attendance marking (manual bulk and per-student), QR scanning, alerts	ADMIN, COACH
F8	Fee management (record, mark paid, waive, receipts, reminders, statuses)	ADMIN
F9	Expense tracking and analysis	ADMIN
F10	Performance assessments (5 rating dimensions + sub-skills)	ADMIN, COACH
F11	Development goals with progress updates	ADMIN, COACH
F12	Training sessions with per-student attendance records	ADMIN, COACH
F13	Match management with full scorecards and MOTM	ADMIN, COACH
F14	Player rankings	ADMIN, COACH
F15	Coaching staff management (invite, assign, deactivate)	ADMIN

#	Function	Primary Actors
F16	User management (create, roles, status, parent–student links, coach–student assignments)	ADMIN
F17	Training packages (public pricing + admin CRUD)	ADMIN, Public
F18	Reports and exports (8 report types; Excel, PDF, CSV)	ADMIN, COACH
F19	Notifications (in-app, push) and announcements	All roles
F20	Audit logs of all system activity	ADMIN
F21	QR code generation and camera-based scanning	ADMIN, COACH
F22	Offline support: read cache, write queue, idempotent replay, background sync	All roles (mobile)
F23	Global search across users/students	All roles

2.3 User Classes and Characteristics

Class	Description	Privileges
Public visitor	Unauthenticated user browsing the website or submitting an admission application	Public pages only
ADMIN	Academy owner/manager	All modules; the only role with access to fees, expenses, packages, users, settings, audit logs, coaches, admissions
COACH	Coaching staff	Management modules scoped to own students/batches/matches (students, attendance, performance, goals, training, matches, rankings, reports, scan)
STUDENT	Player	Read-only access to own profile, attendance, fees, performance, goals, training records, matches, announcements
PARENT	Guardian	Read-only access to linked children’s data (same scope as STUDENT, for multiple children)

2.4 Operating Environment

Aspect	Requirement
Supported browsers	Modern evergreen browsers (Chrome, Edge, Firefox, Safari) — desktop and mobile
Platform	Web; installable as PWA (standalone display mode, portrait orientation)

Aspect	Requirement
Mobile	Responsive layouts: sidebar navigation on desktop, bottom tab navigation + hamburger drawer on mobile
Camera	Required for QR scanning (html5-qrcode); permissions policy restricts camera to self-origin
Offline	Supported via service worker + IndexedDB for read caching and write queueing
Deployment	Vercel (serverless), PostgreSQL on Neon, file storage on Vercel Blob

2.5 Design and Implementation Constraints

- **Framework:** Next.js 16 (App Router), React 19, TypeScript 5, Tailwind CSS 4
- **ORM:** Prisma 7 with @prisma/adapters driver adapter against PostgreSQL (Neon)
- **Authentication:** Custom JWT sessions (HS256 via jose) stored in an httpOnly cookie (yfa_session), 7-day expiry with sliding renewal; no third-party auth provider
- **Password hashing:** bcrypt with 10 salt rounds (bcryptjs)
- **Session revocation:** per-user sessionVersion counter included in the JWT; any password change, role change, or deactivation invalidates all existing sessions
- **Server-side rendering:** public pages are server components fetching live database data; authenticated modules are client components backed by server actions and API routes
- **Access control:** enforced in two layers — middleware (src/proxy.ts) for route-level access and server-side RBAC (src/lib/rbac.ts) for data-level scoping (IDOR protection)
- **Deployment build:** npx prisma migrate deploy && npx prisma generate && next build
- **Security headers:** HSTS (2 years), X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy, Permissions-Policy — applied to all routes; service worker served with no-cache

2.6 Assumptions and Dependencies

- Currency is Pakistani Rupees (PKR) and the timezone is Asia/Karachi throughout the app.
- The academy's contact details (phone, WhatsApp, email, address) are stored in database settings and rendered across public pages.
- Email delivery (password reset) uses Resend; in development without a RESEND_API_KEY, reset links are returned/logged instead of emailed.
- Push notifications require VAPID keys; when absent, the app continues to function with in-app notifications only.
- Fee reminders are generated as pre-filled WhatsApp messages (deep links); no automated WhatsApp sending is implemented.

- The seed script wipes all existing data before seeding and is intended for development/staging only.
 - Photo/document files are stored in Vercel Blob; image optimization allows blob hostnames only.
-

3. System Features and Requirements

Each feature section lists: description, actors, functional flow, business rules, and key data models.

3.1 Authentication and Session Management

Description: Email/password login, logout, and session lifecycle management for all roles.

Actors: All users (ADMIN, COACH, STUDENT, PARENT).

Functional Flow:

1. User visits /login and submits email + password.
2. Server validates credentials against the User record (bcrypt compare).
3. On success, a JWT (HS256, payload includes user ID and sessionVersion) is issued and stored in the yfa_session httpOnly cookie (7-day expiry, sliding renewal on each request).
4. User is redirected to their role's portal: ADMIN → /dashboard, COACH → /coach, STUDENT → /student, PARENT → /parent.
5. On logout, the cookie is destroyed and the user is redirected to /.
6. Authenticated users visiting /login are redirected to their role portal.

Business Rules:

- Only User.status = ACTIVE can authenticate.
- Login is rate-limited (default 10 attempts/minute per client IP, configurable via AUTH_RATE_LIMIT_MAX).
- A failed login does not reveal whether the email exists.
- Passwords are never stored or transmitted in plain text.
- Each successful login creates an audit Activity entry (type LOGIN).
- Session cookies are httpOnly, secure in production, sameSite=lax.
- Any request re-validates the user's sessionVersion; mismatch invalidates the session (revoked sessions are signed out on next request).

Key Models: User, Activity, PasswordResetToken.

3.2 Role-Based Access Control (RBAC)

Description: Route-level and data-level access restriction by role.

Actors: All roles.

Functional Flow:

1. Middleware (`src/proxy.ts`) evaluates the requested path against role permission lists before any page renders.
2. Server actions and API routes additionally enforce role checks and data scoping.

Business Rules (route access):

Path group	Allowed roles
<code>/, /login, /forgot-password, /reset-password, /about, /programs, /coaches, /players, /matches, /gallery, /contact, /apply, /scan/[token], /api/*</code>	Public (no auth)
<code>/dashboard, /students, /attendance, /performance, /dashboard/matches, /rankings, /goals, /training, /reports, /scan</code>	ADMIN, COACH
<code>/fees, /expenses, /dashboard/coaches, /settings, /users, /audit-logs, /admissions, /packages</code>	ADMIN only
<code>/coach</code>	COACH only
<code>/student</code>	STUDENT only
<code>/parent</code>	PARENT only
<code>/notifications, /profile</code>	Any authenticated role
Unknown paths	Render custom 404 page

Data-level scoping (studentScopeWhere):

- ADMIN: all students.
- COACH: students where `coachId = user.id`.
- PARENT: students linked via `StudentParent`.
- STUDENT: own student record (`studentId`).

Business Rules:

- Access to a module the user is not permitted to view redirects to `/forbidden` (or the user's own dashboard).
- Every student-scoped server action verifies the target student is within the caller's scope (`assertStudentAccess`).
- Coaches see financial fields as zeroed/hidden for their students (e.g., `monthlyFee` masked to 0 in API responses).

- ADMINS are the only role that can manage finances, users, settings, and audit data.

Key Models: User, Student, StudentParent.

3.3 Admin Dashboard and Analytics

Description: The ADMIN landing page with academy-wide KPIs, trends, and quick actions.

Actors: ADMIN (also a simplified coach variant for COACH — see §3.27).

Functional Flow:

1. ADMIN visits /dashboard.
2. Filters (month, batch, coach, student status) are applied to the underlying aggregates.
3. Dashboard renders stat cards, charts, and widgets (all computed live from the database).

Dashboard contents:

- Stat cards: total students, active students, attendance %, fee collected, pending fees, overdue fees, monthly income, expenses, net profit, upcoming matches, coaches, batches.
- Charts (Recharts): 6-month trends for students, attendance, income, and expenses; skill-level distribution; batch distribution.
- Widgets: top players (by average rating), upcoming fee dues, recent activity feed, quick actions (add student, mark attendance, record fee, add match, etc.).
- 6-month trend charts for income and expenses vs. month.

Business Rules:

- All figures are computed server-side from live data.
- Fee figures are only visible to ADMIN (dashboard is admin-only).
- Quick actions deep-link into the relevant modules.

Key Models: Student, Attendance, Fee, Performance, Match, Batch, Expense, Activity, User.

3.4 Student Management

Description: Full lifecycle management of students: creation, profiles, photos, documents, activation, deactivation, and soft deletion.

Actors: ADMIN, COACH (scoped to own students).

Functional Flow:

1. ADMIN/COACH opens /students (list) or /students/new (create form).
2. Form captures: name, guardian name, mobile, WhatsApp, date of birth, gender, address, join date, batch, skill level, monthly fee, emergency contact, blood group, photo, playing role, batting/bowling style, jersey number, preferred position.
3. On save, the system generates a YFA-NNNN student ID and a unique QR token, uploads the photo to Vercel Blob, and checks batch capacity.
4. From the list, users can view a student profile (/students/[id]), edit, activate/deactivate, or delete (soft).
5. Documents tab allows uploading/downloading student documents (max 8 MB each).

Business Rules:

- Student ID is auto-incremented in YFA-NNNN format and unique.
- QR token is a unique 24-byte random hex value.
- Batch assignment is rejected if the batch is at capacity.
- Deactivation: student cannot be authenticated/selected for new attendance; if deactivated while in a batch, the batch assignment is cleared.
- Delete is a soft delete (deletedAt set, status → INACTIVE); data remains in the database for audit purposes.
- Mobile numbers are validated with a Pakistani phone-number pattern.
- Only ADMIN/COACH within scope may view or modify students.
- Documents are stored in Vercel Blob; deleting a student document also deletes the stored file.

Key Models: Student, Batch, Document, User.

3.5 Batch Management

Description: Training batch (group) management: name, description, coach, age group, schedule, capacity.

Actors: ADMIN (create/edit/delete), COACH (view own batches).

Functional Flow:

1. ADMIN creates/edits batches from the dashboard or settings: name, description, assigned coach, age group, training days, training time, location, capacity.
2. Batches appear in public pages (programs/contact), attendance, training, and admissions as selectable options.

Business Rules:

- Batch name is unique.
- Deleting a batch is blocked while students are assigned to it.

- Coach assignment links a User (role COACH) to the batch; coaches see only their own batches.
- Inactive batches are excluded from public display.

Key Models: Batch, User, Student.

3.6 Attendance Tracking

Description: Daily attendance marking per student, with statuses, monthly summaries, alerts, and QR-based marking.

Actors: ADMIN, COACH (scoped).

Functional Flow:

1. /attendance shows the selected date with students of a chosen batch (or all scoped).
2. Mark each student PRESENT, ABSENT, LEAVE, LATE, or EXCUSED, then save.
3. Alternatively, bulk-mark all scoped students as ABSENT for a date.
4. Attendance can also be recorded by scanning a student's QR code (see §3.22) or via offline queue replay.
5. History view shows monthly attendance summaries per student.

Business Rules:

- One attendance record per student per date (unique constraint; saves upsert).
- Coaches can only mark attendance for their own students.
- When a student's attendance falls below 75% (rolling), a low-attendance notification is sent to linked parents.
- Attendance percentage feeds dashboards, reports, rankings, and fee reports.
- Marking attendance creates Activity entries (type ATTENDANCE_MARKED).

Key Models: Attendance, Student, Batch, Notification.

3.7 Fee Management

Description: Monthly fee recording, payment tracking, receipts, waivers, and reminders. ADMIN-only.

Actors: ADMIN.

Functional Flow:

1. /fees lists fee records with filters (month, status, batch) and collection summary (collected, due, collection rate, overdue; daily/weekly/monthly collection totals; 6-month trend).

2. Record a fee for a student/month: monthly fee, discount, paid amount, payment method, remarks.
3. System computes balance and status: PAID (balance 0), PARTIAL (0 < paid < total), PENDING (no payment), OVERDUE (past due date), WAIVED.
4. Receipt number generated automatically (RCP-yyyyymm-xxxxxx) on full payment; a notification is sent to the student/parent.
5. Quick “mark paid” and “waive fee” actions available; reminders generate a pre-filled WhatsApp message for the guardian.

Business Rules:

- One fee record per student per month (YYYY-MM).
- Only ADMIN can manage fees; coaches never see financial data.
- Fee statuses are auto-computed, not manually set.
- Waived fees record the waiving admin.
- Fee records feed the financial summary report, upcoming-dues widget, and parent/student portals.

Key Models: Fee, Student, Notification, Activity.

3.8 Expense Tracking

Description: Recording and analysis of academy operating expenses. ADMIN-only.

Actors: ADMIN.

Functional Flow:

1. /expenses lists expenses with filters and category breakdown, totals, and a 6-month trend.
2. Add/edit/delete expense: title, category (EQUIPMENT, GROUND, TRANSPORT, UTILITIES, SALARIES, MAINTENANCE, EVENTS, OTHER), amount, date, payment method, notes.

Business Rules:

- Only ADMIN can create, edit, or delete expenses.
- Expense data feeds the admin dashboard (monthly expenses, net profit) and the financial summary report.
- Deletion is logged to the audit trail.

Key Models: Expense, Activity.

3.9 Performance Assessments

Description: Periodic ratings of each student across five dimensions with optional sub-skill breakdowns.

Actors: ADMIN, COACH (scoped).

Functional Flow:

1. /performance shows assessment history with previous-rating comparison for trend.
2. Add an assessment: date, ratings (1–10) for batting, bowling, fielding, fitness, discipline, optional sub-skills JSON, remarks.
3. Overall rating is auto-computed as a weighted average; on save, the ADMIN is notified.

Business Rules:

- Ratings are integers 1–10.
- Overall rating auto-computed; stored as a float.
- Coaches may only assess their own students.
- Assessments feed player profiles, rankings, reports, and portals.
- Updates/deletes are IDOR-guarded.

Key Models: Performance, Student, User, Notification.

3.10 Development Goals

Description: Per-student training goals with measurable progress and update history.

Actors: ADMIN, COACH (scoped).

Functional Flow:

1. /goals lists goals with students and assigned coaches.
2. Create a goal: student, title, description, category, baseline, target, progress (0–100), status, deadline.
3. Progress updates create GoalUpdate records (progress % + note).
4. Progress reaching 100 auto-marks the goal ACHIEVED.

Business Rules:

- Goal statuses: NOT_STARTED, IN_PROGRESS, ACHIEVED, CANCELLED.
- Each progress change is timestamped and attributed to the creator.
- Coaches scoped to own students.
- Goals appear on student and parent portals.

Key Models: Goal, GoalUpdate, Student, User.

3.11 Training Sessions

Description: Scheduled group training sessions with per-student attendance records and highlights.

Actors: ADMIN, COACH (scoped).

Functional Flow:

1. /training lists sessions (date, batch, coach, topic, category, time, location).
2. Create/edit/delete sessions; on creation the ADMIN is notified.
3. For each session, record per-student presence, notes, and highlights (unique per session+student).

Business Rules:

- Session categories: BATTING, BOWLING, FIELDING, FITNESS, WICKETKEEPING, TACTICAL, MATCH_PRACTICE.
- Deleting a session deletes its records.
- Training records appear on student/parent portals (recent training history).
- Coaches scoped to own batches/students.

Key Models: TrainingSession, TrainingSessionRecord, Batch, Student, User.

3.12 Match Management and Scorecards

Description: Management of matches and complete per-player scorecards.

Actors: ADMIN, COACH (scoped to own matches/students).

Functional Flow:

1. /dashboard/matches lists matches (past, upcoming) with results.
2. Create/edit a match: date, opponent, venue, match type (FRIENDLY, TOURNAMENT, LEAGUE, PRACTICE, OTHER), competition, toss, overs, notes, result (WON, LOST, DRAW, TIE), assigned coach.
3. For each match, enter per-player records: selected, batting position, runs, balls faced, 4s, 6s, dismissal type, wickets, overs bowled, maidens, runs conceded, catches, run outs, stumpings; computed strike rate and economy; MOTM flag.
4. Public /matches and /matches/[id] pages render fixtures, results, and full scorecards.

Business Rules:

- One record per student per match (unique).

- Dismissal types: BOWLED, CAUGHT, LBW, RUN_OUT, STUMPED, NOT_OUT, RETIRED, OTHER.
- Strike rate and economy are computed from entered data.
- Coaches only manage matches they created or that involve their students.
- Public pages never reveal contact/private data — only cricket stats.
- Match data feeds the public site, admin dashboard, and reports.

Key Models: Match, MatchRecord, Student, User.

3.13 Player Rankings

Description: Ranking of students by performance metrics.

Actors: ADMIN, COACH.

Functional Flow:

1. /rankings computes rankings from average overall rating, assessment count, and attendance.
2. Rankings are exportable (Excel) and feed the public “top players” widget.

Business Rules:

- Rankings include only scoped students (coaches see own students).
- Ranking order: average rating primary, tie-broken by assessment count and attendance.

Key Models: Performance, Student, Attendance.

3.14 Coaching Staff Management

Description: Management of coach accounts, specializations, and student/batch assignments.

Actors: ADMIN.

Functional Flow:

1. /dashboard/coaches lists coaches with specialization, assigned students, assigned batches, status.
2. Invite a coach (creates account with temporary password) or create via /users.
3. Assign coaches to batches and bulk-assign coaches to students.
4. Deactivate/reactivate coaches.

Business Rules:

- Coach accounts are User records with role COACH plus a CoachProfile (specialization).
- Deactivating a coach prevents login; their batches/students are not deleted.
- Assigning a coach to a student replaces the student's coachId.
- Only ADMIN manages coaches.

Key Models: User, CoachProfile, Student, Batch.

3.15 User Management

Description: Creation and management of all login accounts (coaches, parents, students).

Actors: ADMIN.

Functional Flow:

1. /users lists all users with role, status, and related links.
2. Create a user: full name, email, mobile, role (COACH/PARENT/STUDENT), temporary password, and role-specific links (coach → batches; parent → linked students; student → linked student record).
3. Change roles, activate/deactivate accounts, and bulk-assign coaches/link parents.

Business Rules:

- Email is unique.
- New users receive a temporary password that they should change.
- Changing a role or deactivating a user increments sessionVersion, revoking all existing sessions.
- The last remaining ADMIN cannot be deactivated or demoted (last-admin protection).
- Password-reset links can be generated for any user by an ADMIN.

Key Models: User, StudentParent, Student, Batch, PasswordResetToken.

3.16 Admission Pipeline

Description: Public admission application → admin review → conversion to a student record.

Actors: Public (apply), ADMIN (review/convert).

Functional Flow (public):

1. Visitor opens /apply, reviews the 3-step process, and submits the form: student name, DOB, gender, guardian name, phone, email, preferred batch, experience, playing role, message.

2. Application is stored with status NEW; a confirmation is shown.
3. Submission is rate-limited (5 per 15 minutes per client).

Functional Flow (admin):

1. /admissions lists applications with statuses and preferred batches.
2. Review: move through NEW → REVIEW → APPROVED / REJECTED.
3. Convert an APPROVED application: creates a Student record (auto YFA-NNNN ID) linked to the preferred batch.

Business Rules:

- Admission statuses: NEW, REVIEW, APPROVED, REJECTED, CONVERTED.
- Conversion requires prior approval and records the resulting student ID.
- Only ADMIN reviews/converts admissions.
- Active batches are shown in the public form.

Key Models: Admission, Batch, Student.

3.17 Training Packages

Description: Publicly displayed training packages with pricing, billing cycles, and feature lists.

Actors: ADMIN (CRUD), Public (view).

Functional Flow:

1. ADMIN manages packages in /packages: name, description, price, billing type (MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY), sessions per week, feature list, active flag, start/end dates.
2. Active packages appear on the public homepage, /programs, and /programs/[id] (detail page with Apply CTA).

Business Rules:

- Only active packages are shown publicly.
- Deleted/toggled packages disappear from public pages immediately.
- Package detail pages generate SEO metadata and 404 for inactive/unknown packages.

Key Models: Package.

3.18 Reports and Exports

Description: Eight report types generated from aggregated live data, exportable in multiple formats.

Actors: ADMIN, COACH (scoped; financial reports ADMIN-only).

Functional Flow:

1. /reports selects report type and filters (date range, fee month, batch).
2. Data is fetched from /api/reports and rendered/exported.

Report matrix:

Report	Formats	Admin-only columns/data
Students	Excel, PDF, CSV	Monthly fee, fees paid/due
Attendance Daily	Excel, CSV, PDF	—
Expenses	Excel	Entire report (ADMIN)
Matches	Excel	—
Fees	Excel	Entire report (ADMIN)
Performance	Excel	—
Rankings	Excel	—
Financial Summary	PDF	Entire report (ADMIN)

Business Rules:

- PDF exports use a branded header (navy/green/gold academy colors).
- Export events are logged to the audit trail (logReportExportAction).
- Coach reports are scoped to own students.
- Fee-month filter applies to fee-based reports.

Key Models: Aggregated views over Student, Attendance, Fee, Expense, Performance, Match, MatchRecord.

3.19 Notifications and Announcements

Description: In-app notifications with optional web push delivery, and targeted announcements.

Actors: All roles (receive/view), ADMIN/COACH (create announcements).

Functional Flow:

1. /notifications lists unread/read notifications with mark-read and delete actions.
2. System generates notifications on events (fee paid, low attendance, performance added, training created, etc.).

3. ADMIN (or authorized) creates announcements targeted at a role or a batch; recipients receive in-app notifications.
4. Push subscriptions (VAPID) deliver notifications to browsers/installed PWA; expired subscriptions are auto-cleaned.

Business Rules:

- Notifications can target a specific user, a role, or (via announcements) a batch.
- Push subscribe endpoint rate-limited (10/min); push sending uses VAPID keys.
- Unread counts appear in the topbar.

Key Models: Notification, Announcement, PushSubscription, Batch, User.

3.20 Audit Logs

Description: Complete audit trail of system activity.

Actors: ADMIN (view).

Functional Flow:

1. Every significant action writes an Activity record (user, type, action, entity, entity ID, details, timestamp).
2. /audit-logs lists activities with filters; used for accountability and troubleshooting.

Business Rules:

- 37 activity types cover: login/logout, password reset, student CRUD, attendance marking, fee events, performance, goals, matches, training, admissions, expenses, packages, coaches, announcements, settings, backups, notifications, and OTHER.
- Activity records are append-only (never edited/deleted by the UI).

Key Models: Activity, User.

3.21 Password Reset

Description: Self-service password recovery and authenticated password change.

Actors: All users (self-service), ADMIN (generate reset links).

Functional Flow (forgot password):

1. User requests reset on /forgot-password with their email.
2. System generates a random 32-byte token, stores a bcrypt hash in PasswordResetToken (1-hour expiry).
3. An email with a reset link is sent (Resend); in development without an API key, the link is returned/logged.

4. User opens `/reset-password?token=...`, sets a new password (min 8 chars, letter + number, must match confirmation).
5. Token is matched by bcrypt comparison, marked used; `sessionVersion` increments (revokes other sessions).

Functional Flow (change password):

1. Authenticated user provides current + new password from profile.
2. Current password verified; new password hashed; session re-issued.

Business Rules:

- Reset tokens expire after 1 hour and are single-use.
- Password policy: minimum 8 characters, at least one letter and one number.
- Reset flow is rate-limited.
- The email response never reveals whether the account exists.

Key Models: PasswordResetToken, User, Activity.

3.22 QR Code System

Description: Per-student QR codes for fast attendance marking.

Actors: ADMIN, COACH (scanned), anyone with the student's card (verification), ADMIN/COACH (scan).

Functional Flow:

1. Each student's QR token generates a scannable code (SVG/data URL) printed on the student's card.
2. Scanning the code opens the public page `/scan/[token]`, which verifies the token.
3. An authenticated ADMIN/COACH scans codes with the camera (`/scan`) to mark today's attendance (`POST /api/scan/[token]`, rate-limited 30/min).

Business Rules:

- Tokens are unique, 24-byte random hex, and unguessable.
- Verification checks the student exists and is not deleted.
- Coach scanning is scoped to their own students.
- The public verification page shows student info and today's attendance status but no financial data.

Key Models: Student, Attendance, Activity.

3.23 Public Website Pages

Description: The public marketing surface of the academy.

Actors: Public visitors.

Pages:

Route	Content (live data)
/	Hero, live stats (active students/coaches/batches/matches), value cards, programs pillars, facilities, achievements (aggregated runs/wickets/catches + top 5 performers), admission process, packages, FAQ, contact CTA; JSON-LD structured data
/about	Coaching approach, values, player development, facilities, CTA
/programs	Active packages with pricing/features + current training batches with schedule/coach
/programs/[id]	Package detail, features, Apply CTA; dynamic metadata; 404 for inactive
/coaches	Active coaches: photo, name, specialization, assigned batches
/players	Active students: photo, name, student ID, skill level, batch, aggregate runs/wickets/catches
/players/[id]	Player profile: stats cards (runs, wickets, catches, MOTM), roles, styles, recent assessments; dynamic metadata; 404 for inactive
/matches	All matches: opponent, date, result badge, type, venue, player count
/matches/[id]	Scorecard: team runs/wickets/catches, full per-player table (runs, balls, 4s, 6s, dismissal, wickets, overs, catches, MOTM), notes; dynamic metadata
/gallery	Student photos grid (students with photoUrl) with name + skill level
/contact	Phone, WhatsApp (pre-filled message), email, address, training/office hours from settings
/apply	Admission form with active batch preferences (see §3.16)

Business Rules:

- All pages are server components fetching live database data.
- Public pages never expose financial data, contact details, or guardian information.
- Shared navigation: landing-header (8 links with active state), landing-footer (Quick Links / Academy / Account groups, session-aware), mobile-menu (hamburger drawer), breadcrumbs on detail pages.
- /admission redirects permanently to /apply.
- The header/footer account section is session-aware (Login vs. Dashboard / My Profile).

Key Models: Student, CoachProfile, Batch, Match, MatchRecord, Package, Setting, Performance.

3.24 SEO and Structured Data

Description: Search-engine optimization across the public site.

Functional Flow:

- Global metadata: title template %s | Young Fighters Academy, description, OpenGraph, Twitter cards, robots (index, follow), icons, apple-web-app tags, theme colors (light #145c38, dark #16263d).
- Dynamic metadata on /programs/[id], /players/[id], /matches/[id] (package name, player name, opponent).
- JSON-LD SportsActivityLocation schema on the homepage (name, description, phone, email, address PK, URL, price range).
- sitemap.ts (dynamic): /, /about, /programs, /coaches, /players, /matches, /gallery, /apply, /contact, /login with priorities and change frequencies.
- robots.ts: allows public paths; disallows all authenticated paths and /api/.

Business Rules:

- Authenticated routes are excluded from sitemap and robots.
 - Metadata base URL is the production Vercel alias.
-

3.25 PWA and Offline Support

Description: Installable app with offline read caching, offline write queueing with idempotent replay, and web push.

Actors: All roles on browsers/PWA installs.

Service worker (/sw.js):

- Pre-caches app shell (/, /login, manifest, icons).
- Navigation: network-first, falls back to cache, then /.
- Static assets: cache-first.
- Background sync (yfa-sync tag) wakes clients to flush the offline queue.
- Push event handling: shows notifications with icon/badge/vibration/URL; click focuses/navigates.

Offline write queue (connectivity-provider):

- IndexedDB stores write-queue (attendance marks, fee records) and read-cache (API responses).
- On reconnect (online event, 30-second poll, or SW background sync), queued writes POST to /api/offline/replay (rate-limited 60/min).

- Idempotency: each write carries an entryId; the server stores processed IDs in OfflineSyncRecord to prevent duplicates.
- Entries with 5+ failed attempts surface a “failed to sync” warning.

Business Rules:

- Supported offline actions: attendance.mark, attendance.bulk, fee.record (fees admin-only).
- Replay validates scope server-side.
- Web manifest: name “Young Fighters Academy”, short name “YFA”, standalone display, portrait orientation, 192/512 icons (any + maskable).

Key Models: OfflineSyncRecord, PushSubscription.

3.26 Settings and Global Configuration

Description: Key-value academy settings rendered across the site.

Actors: ADMIN (manage), all roles (consume).

Functional Flow:

1. /settings (ADMIN) edits key-value settings: academy name, phone, email, address, hours, receipt footer.
2. Public pages (contact, footer, homepage), receipts/PDFs, and reports consume these values.

Business Rules:

- Keys are unique; values are strings.
- Settings updates are logged to the audit trail.

Key Models: Setting, Activity.

3.27 Role-Specific Portals

Description: Landing dashboards for COACH, STUDENT, and PARENT roles.

COACH portal (/coach):

- My students, my batches, today’s attendance status per student, recent performance assessments, upcoming matches, 7-day attendance bar chart, recent activity feed.

STUDENT portal (/student):

- Personal info, batch, attendance history (last 90 days), fees (last 6 months), performance (last 6), match records (last 10), development goals (last 10), training records (last 10), upcoming matches, announcements.

PARENT portal (/parent):

- All linked children, each with: batch, attendance (last 60 days), fees (last 6 months), performance (last 6), match records, goals (last 3), training records (last 10), announcements.

Business Rules:

- Portals are read-only for STUDENT and PARENT.
- PARENTs see only linked children; the parent page includes a child selector when multiple children are linked.
- Portals never expose other students' data or financial totals beyond the child's own fees.

Key Models: Student, StudentParent, Attendance, Fee, Performance, Goal, MatchRecord, TrainingSessionRecord, Batch, Announcement.

3.28 Global Search

Description: Cross-module search from the topbar.

Actors: All authenticated roles.

Functional Flow:

1. Topbar search field (`global-search.tsx`) queries students (name, ID, guardian, mobile, batch) and users via `searchStudents` (scope-aware).
2. Results deep-link to the matching profile.

Business Rules:

- Coaches only see results within their scope.
- Financial fields are excluded from search results for coaches.

Key Models: Student, User, Batch.

4. Database Schema

4.1 Enumerations (18)

Enum	Values
Role	ADMIN, COACH, STUDENT, PARENT
UserStatus	ACTIVE, INACTIVE

Enum	Values
StudentStatus	ACTIVE, INACTIVE
Gender	MALE, FEMALE, OTHER
SkillLevel	BEGINNER, INTERMEDIATE, ADVANCED, PROFESSIONAL
AttendanceStatus	PRESENT, ABSENT, LEAVE, LATE, EXCUSED
FeeStatus	PAID, PARTIAL, PENDING, OVERDUE, WAIVED
PaymentMethod	CASH, BANK_TRANSFER, JAZZCASH, EASYPAISA, CARD, OTHER
MatchResult	WON, LOST, DRAW, TIE
MatchType	FRIENDLY, TOURNAMENT, LEAGUE, PRACTICE, OTHER
Dismissal	BOWLED, CAUGHT, LBW, RUN_OUT, STUMPED, NOT_OUT, RETIRED, OTHER
GoalStatus	NOT_STARTED, IN_PROGRESS, ACHIEVED, CANCELLED
TrainingCategory	BATTING, BOWLING, FIELDING, FITNESS, WICKETKEEPING, TACTICAL, MATCH_PRACTICE
AdmissionStatus	NEW, REVIEW, APPROVED, REJECTED, CONVERTED
ExpenseCategory	EQUIPMENT, GROUND, TRANSPORT, UTILITIES, SALARIES, MAINTENANCE, EVENTS, OTHER
BillingType	MONTHLY, QUARTERLY, HALF_YEARLY, YEARLY
AnnouncementPriority	LOW, MEDIUM, HIGH
ActivityType	LOGIN, LOGOUT, PASSWORD_RESET, STUDENT_CREATED, STUDENT_UPDATED, STUDENT_DEACTIVATED, STUDENT_ACTIVATED, STUDENT_DELETED, ATTENDANCE_MARKED, FEE_RECORDED, FEE_UPDATED, RECEIPT_GENERATED, PERFORMANCE_ADDED, PERFORMANCE_UPDATED, GOAL_ADDED, GOAL_UPDATED, MATCH_ADDED, MATCH_UPDATED, TRAINING_SESSION_CREATED, TRAINING_ATTENDANCE_RECORDED, ADMISSION_SUBMITTED, ADMISSION_REVIEWED, EXPENSE_ADDED, EXPENSE_UPDATED, EXPENSE_DELETED, PACKAGE_CREATED, PACKAGE_UPDATED, PACKAGE_DELETED, COACH_ADDED, COACH_UPDATED, COACH_DEACTIVATED, ANNOUNCEMENT_CREATED, SETTINGS_UPDATED, BACKUP_EXPORTED, BACKUP_RESTORED, NOTIFICATION_SENT, OTHER

4.2 Models (25)

Model	Key fields (type)	Relations / notes
User	id, email (unique), passwordHash, fullName, mobile?, role, status, photoUrl?, studentId?,	1:1 CoachProfile; M:N Student via StudentParent/StudentCoach; batches, performances, matches, goals, trainingSessions, activities,

Model	Key fields (type)	Relations / notes
Batch	sessionVersion, lastLoginAt?, createdAt, updatedAt id, name (unique), description?, coachId?, ageGroup?, trainingDays?, trainingTime?, trainingLocation?, capacity, isActive	notifications, resetTokens, pushSubscriptions, uploadedDocuments Coach; students; sessions; admissions; announcements
Student	id, studentId (unique YFA- NNNN), fullName, guardianName, mobile, whatsapp?, dob, gender, address?, joinDate, batchId?, skillLevel, monthlyFee, emergencyContact?, bloodGroup?, photoUrl?, status, email?, playingRole?, battingStyle?, bowlingStyle?, jerseyNumber?, preferredPosition?, coachId?, qrToken (unique), deletedAt? (soft delete)	parentLinks; attendance; fees; performances; matchRecords; goals; trainingRecords; documents
CoachProfile	id, userId (unique), specialization?	1:1 with User
StudentParent	id, studentId, parentId	Unique [studentId, parentId]
Attendance	id, studentId, date, status, markedBy	Unique [studentId, date]
Fee	id, studentId, month (YYYY- MM), monthlyFee, discount, paidAmount, balance, dueDate, paymentDate?, paymentMethod?, receiptNumber? (unique), status, remarks?, waivedBy?, createdBy, updatedBy?	Unique [studentId, month]
Performance	id, studentId, date, battingRating,	Index [studentId, date]

Model	Key fields (type)	Relations / notes
	bowlingRating, fieldingRating, fitnessRating, disciplineRating, overallRating, subSkills? (Json), remarks?, coachId?	
Goal	id, studentId, coachId?, title, description?, category?, baseline?, target?, progress, status, deadline?, createdBy?	updates (GoalUpdate[])
GoalUpdate	id, goalId, progress, note?, createdBy, createdAt	—
TrainingSession	id, date, batchId?, coachId?, topic, category, startTime?, endTime?, location?, notes?, createdBy?	records (TrainingSessionRecord[])
TrainingSessionRecord	id, sessionId, studentId, present, notes?, highlights?	Unique [sessionId, studentId]
Admission	id, studentName, dob, gender, guardianName, phone, email?, preferredBatchId?, experience?, playingRole?, message?, status, reviewedBy?, reviewedAt?, studentId?	Index [status], [createdAt]
Match	id, matchDate, opponent, venue?, matchType?, competition?, tossWon?, overs?, notes?, result?, coachId?, createdBy?	records (MatchRecord[])
MatchRecord	id, matchId, studentId, selected, battingPosition?, runs, ballsFaced?, fours, sixes, dismissal?, wickets, oversBowled?, maidens, runsConceded?, catches, runOuts, stumpings, strikeRate?, economy?, manOfTheMatch	Unique [matchId, studentId]
Expense	id, title, category, amount,	Index [date, category]

Model	Key fields (type)	Relations / notes
	date, paymentMethod?, notes?, createdBy, updatedBy?	
Package	id, name, description?, price, billingType, sessionsPerWeek, features (String[]), startDate?, endDate?, isActive, createdBy?	Index [isActive]
Document	id, studentId, title, type?, url, uploadedBy, createdAt	Index [studentId]
Activity	id, userId, type, action, entity?, entityId?, details?, createdAt	Index [createdAt]
Notification	id, userId?, role?, title, body, type, read, createdAt	Index [userId, read], [role]
Announcement	id, title, body, audience, priority, batchId?, createdBy, createdAt	Index [batchId]
PasswordResetToken	id, userId, tokenHash (unique), expiresAt, usedAt?, createdAt	1-hour expiry
Setting	id, key (unique), value, updatedBy?, updatedAt	Academy config
PushSubscription	id, userId, endpoint (unique), keys? (Json), createdAt	Web Push
OfflineSyncRecord	id, key (unique), createdAt	Idempotent offline replay dedupe

5. External Interfaces

5.1 REST API Endpoints

Method	Route	Access	Purpose
GET	/api/students	ADMIN, COACH	Paginated student list with filters (batch, skill, status, gender, coach); includes batches + coaches for filter UI; coaches see monthlyFee: 0
GET	/api/students/[id]	ADMIN, COACH	Student detail with attendance summary, fees (admin only), performance history, match records, upcoming fees; RBAC via assertStudentAccess

Method	Route	Access	Purpose
GET	/api/attendance	ADMIN, COACH	Students + attendance status for a date; optional monthly summary by status
GET	/api/fees	ADMIN	Paginated fees with summary, collection totals (daily/weekly/monthly), 6-month trend
GET	/api/expenses	ADMIN	Paginated expenses with category breakdown, totals, 6-month trend
GET	/api/matches	ADMIN, COACH	Matches + students + upcoming matches, role-scoped
GET	/api/performance	ADMIN, COACH	Paginated performance records with previous rating (trend)
GET	/api/packages	ADMIN	Paginated packages with active count
GET	/api/reports	ADMIN, COACH	Aggregated data for all report types
GET	/api/scan/[token]	ADMIN, COACH	QR token lookup: student info + today's attendance status (coach-scoped)
POST	/api/scan/[token]	ADMIN, COACH	Mark attendance via QR (upsert); rate-limited 30/min
POST	/api/push-subscribe	Any auth	Upsert push subscription; rate-limited 10/min
POST	/api/offline/replay	Any auth	Idempotent offline write replay (attendance.mark, attendance.bulk, fee.record); rate-limited 60/min

5.2 Server Actions (src/app/actions/)

Module	Actions
auth.ts	loginAction, signOut, forgotPasswordAction, resetPasswordAction, changePasswordAction, updateProfileAction, getResetLinkForUser (admin)
students.ts	searchStudents, createStudentAction, updateStudentAction, setStudentStatusAction, deleteStudentAction
attendance.ts	markAttendanceAction, bulkMarkAbsentAction
fees.ts	recordFeeAction, markFeePaidAction, waiveFeeAction, sendFeeReminderAction
expenses.ts	addExpenseAction, updateExpenseAction, deleteExpenseAction
performance.ts	addPerformanceAction, updatePerformanceAction, deletePerformanceAction, saveMatchAction
goals.ts	addGoalAction, updateGoalAction, updateGoalProgressAction, deleteGoalAction, getGoalsData
training.ts	createTrainingSessionAction, updateTrainingSessionAction,

Module	Actions
	deleteTrainingSessionAction, saveTrainingAttendanceAction, getTrainingData
packages.ts	createPackageAction, updatePackageAction, togglePackageAction, deletePackageAction
users.ts	createUserAction, updateUserRoleAction, setUserStatusAction, assignCoachToStudentsAction, linkParentStudentsAction
misc.ts	markNotificationsReadAction, markNotificationReadAction, deleteNotificationAction, createAnnouncementAction, removeStudentFromBatchAction, createBatchAction, updateBatchAction, deleteBatchAction, updateSettingsAction, inviteCoachAction, deactivateCoachAction, reactivateCoachAction, logReportExportAction
documents.ts	uploadStudentDocumentAction (max 8 MB), deleteStudentDocumentAction
admissions.ts	submitAdmissionAction (rate-limited 5/15 min), reviewAdmissionAction, convertAdmissionAction, deleteAdmissionAction, getAdmissionsData, getAdmissionBatches

All actions validate input via Zod schemas (src/lib/validation/schemas.ts, 17+ schemas) and enforce role + scope server-side.

5.3 External Services

Service	Usage	Configuration
Neon PostgreSQL	Primary database via Prisma driver adapter (pg)	DATABASE_URL
Vercel Blob	Student photos and documents	BLOB_READ_WRITE_TOKEN
Resend	Password-reset email delivery (dev fallback logs link)	RESEND_API_KEY
Web Push (VAPID)	Browser push notifications	NEXT_PUBLIC_VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY
WhatsApp	Pre-filled message deep links (fee reminders, contact)	none (deep-link only)

6. Non-Functional Requirements

6.1 Security

Requirement	Specification
Password storage	bcrypt, 10 rounds; never plain text

Requirement	Specification
Session tokens	JWT HS256, httpOnly cookie, 7-day expiry, sliding renewal; secret from AUTH_SECRET
Session revocation	sessionVersion check on every request; revoked on password change, role change, deactivation
Password policy	Min 8 chars, at least one letter + one number
Reset tokens	32-byte random, bcrypt-hashed at rest, 1-hour expiry, single use
Rate limiting	Login (10/min, configurable), reset flow, QR scan POST (30/min), push subscribe (10/min), offline replay (60/min), admission submit (5/15 min)
Access control	Middleware route checks + server-side RBAC scoping (IDOR protection) on every action/API
Sensitive data masking	Coaches see zeroed financial fields; public pages exclude guardian/contact/financial data
Headers	HSTS (2 yr), nosniff, X-Frame-Options DENY, strict Referrer-Policy, Permissions-Policy (camera self-only)
Files	Student documents capped at 8 MB; stored in private Vercel Blob
Last-admin protection	The final ADMIN cannot be deactivated or demoted

6.2 Performance

- Public pages are server-rendered with live DB queries; aggregation occurs in SQL/Prisma, not in the browser.
- Client-side tables use TanStack Table with server-side pagination.
- Images are optimized by Next.js (blob hostnames allow-listed).
- Rate limiting and caching of session verification per request (React.cache).

6.3 Availability and Reliability

- Deployed on Vercel serverless infrastructure with the production alias `https://young-fighters-academy.vercel.app`.
- Database is hosted on Neon (managed PostgreSQL, auto-backups).
- Offline resilience: PWA cache + IndexedDB queue with idempotent replay and stuck-entry surfacing.
- Push subscription auto-cleanup for expired endpoints (404/410).

6.4 Scalability

- Stateless application server; scaling is horizontal via Vercel functions.
- Rate limiter is in-memory/process-local (per-instance protection); database remains the system of record.
- Indexed columns support the primary query patterns (dates, statuses, scopes, unique constraints).

6.5 Usability and Accessibility

- Responsive layouts: desktop sidebar, mobile bottom-nav + hamburger drawer.
- Dark/light theme (next-themes) with system preference; brand palette (green/navy/gold) tuned for contrast in both modes.
- Loading skeletons, empty states, toasts, and confirm dialogs throughout.
- Form validation with inline error messages (react-hook-form + Zod).
- Semantic HTML, aria-labels on icon-only controls, keyboard-accessible menus.
- SEO metadata on every public page; JSON-LD for rich results.

6.6 Maintainability

- TypeScript strict; Prisma-generated types; Zod schemas as the single validation source.
- Centralized: auth (src/lib/auth.ts), RBAC (src/lib/rbac.ts), session (src/lib/session.ts), utils (src/lib/utils.ts), UI primitives (src/components/ui/).
- ESLint enforced; CI build runs prisma migrate deploy && prisma generate && next build.
- Audit logging covers all mutating operations.

7. Appendix

7.1 Technology Stack

Layer	Technology
Framework	Next.js 16 (App Router), React 19, TypeScript 5
Styling	Tailwind CSS 4, clsx + tailwind-merge
Database	PostgreSQL (Neon) via Prisma 7 + @prisma/adapter-pg
Auth	Custom JWT (jose HS256), bcryptjs, httpOnly cookie
Forms/validation	react-hook-form, zod 4, @hookform/resolvers
Tables/charts	TanStack Table, Recharts
Reports	jspdf + jspdf-autotable, xlsx
QR	qrcode (generate), html5-qrcode (scan)
PWA	Custom service worker, idb (IndexedDB), web-push (VAPID)
Storage	@vercel/blob
Email	Resend (REST)
Notifications	Web Push + in-app notification store
Testing (dev)	Vitest, Testing Library, jsdom
Deployment	Vercel (serverless), Neon, Vercel Blob

7.2 Environment Variables

Variable	Purpose
----------	---------

Variable	Purpose
DATABASE_URL	Neon PostgreSQL connection string
AUTH_SECRET	JWT signing secret (HS256)
APP_URL	Application base URL (QR links, emails)
RESEND_API_KEY	Password-reset email delivery (optional; dev logs link)
BLOB_READ_WRITE_TOKEN	Vercel Blob storage token
NEXT_PUBLIC_VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY	Web Push keys (optional)
SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD	Override seed admin credentials (dev only)
AUTH_RATE_LIMIT_MAX	Login rate limit per minute (default 10)

7.3 Seed Data (development reference)

- 5 users: ADMIN (admin@yfa.pk), 2 coaches, 1 parent, 1 student login
- 2 batches (Morning: Mon/Wed/Fri 6–8 AM; Evening: Tue/Thu/Sat 4–6 PM)
- 6 students (YFA-001...006, mixed skill levels, monthly fees Rs 6,000–12,000)
- 60 days of attendance, 3 months of fees, 4 performance assessments per student, 3 matches with full scorecards, 7 expenses, 6 settings, 1 announcement

End of SRS — Young Fighters Academy, version 1.0. Compiled from the live production codebase.